

Actividad N5: Manual Programador

Sofia Salaberry, Martina Araujo, Thiago Goya

1

Participantes:

Sofia Salaberry, Martina Araujo y Thiago Goya

Escuela:

Escuela Técnica N°1 de Vicente Lopez

Curso:

Ciclo Superior 5°3°

Docentes a cargo:

Yamil Ganduglia, York Elias Mansilla Muñoz

Materia:

Lab. Programación

Repositorio:

Sofia: https://github.com/sopita122/Datos_Estructurados.git

Año: 2026

Actividad N5: Manual Programador

Sofia Salaberry, Martina Araujo, Thiago Goya

2

Estructura utilizada en el programa

```
struct EntidadProyecto {  
    int id;  
    char nombre[50];  
    float metrica;  
};
```

Los elementos que forman la estructura son:

id: identificador de la entidad.

nombre: nombre o descripción de la entidad.

metrica: valor decimal relacionado con la entidad.

Objetivo del manual

El objetivo de este manual es explicar cómo está desarrollado el programa de la Actividad 5 y cómo se utilizan las estructuras (struct) y los punteros en C++.

El programa permite modelar una entidad mediante una estructura llamada EntidadProyecto, ingresar sus datos mediante una función y acceder a ellos utilizando un puntero y el operador flecha (->).

También se explica el uso de `cin.ignore()` y `cin.getline()` para ingresar correctamente datos de texto.

¿Qué es un struct?

Un struct permite agrupar diferentes variables relacionadas dentro de una misma estructura lógica.

Creación de la entidad

En `main()` se crea la siguiente entidad:

```
EntidadProyecto miEntidad = {0, "Vacio - [Su Nombre y Apellido]",  
0.0f};
```

La variable comienza con valores iniciales y luego se modifica con `cargarDatos()`.

Actividad N5: Manual Programador

Sofia Salaberry, Martina Araujo, Thiago Goya

3

La estructura se crea una sola vez en main() y se utiliza durante todo el programa.

Puntero a un struct

Un puntero almacena una dirección de memoria.

La dirección se obtiene mediante el operador &:

```
cargarDatos(&miEntidad);
```

La función recibe esa dirección:

```
void cargarDatos(EntidadProyecto* ptr);
```

Por lo tanto, ptr contiene la dirección de miEntidad.

Operador flecha (->)

Con un puntero a un struct se utiliza el operador -> para acceder a sus miembros:

```
ptr->id  
ptr->nombre  
ptr->metrica
```

Por ejemplo:

```
cin >> ptr->id;
```

Esta instrucción modifica el id original.

Cuando se trabaja con una variable directa se utiliza el punto:

```
miEntidad.id
```

En cambio, cuando se trabaja con un puntero se utiliza:

```
ptr->id
```

Se utiliza -> porque ptr es un puntero y no una variable directa.

Pasaje de la dirección mediante puntero

La función utilizada es:

```
void cargarDatos(EntidadProyecto* ptr)
```

Y se llama de la siguiente manera:

```
cargarDatos(&miEntidad);
```

De esta forma, la función trabaja sobre la estructura original.

Por ejemplo:

Actividad N5: Manual Programador

Sofia Salaberry, Martina Araujo, Thiago Goya

4

```
ptr->id = 10;
```

Esta modificación también cambia el valor de miEntidad.

No es necesario crear otra estructura.

Carga de datos

```
void cargarDatos(EntidadProyecto* ptr) {
    cout << "--- INGRESO DE DATOS MEDIANTE OPERADOR FLECHA ---" <<
endl;
    cout << "=> Ingrese el ID (entero): ";
    cin >> ptr->id;
    cin.ignore();

    cout << "=> Ingrese el Nombre: ";
    cin.getline(ptr->nombre, 50);

    cout << "=> Ingrese la Metrica (float): ";
    cin >> ptr->metrica;
}
```

La función solicita tres datos: ID, Nombre y Métrica. Cada dato se guarda directamente en la estructura.

Buffer de entrada: cin.ignore() y cin.getline()

Después de ejecutar:

```
cin >> ptr->id;
```

queda el Enter (\n) en el buffer.

Por eso se utiliza:

```
cin.ignore();
```

antes de:

```
cin.getline(ptr->nombre, 50);
```

Esto permite leer correctamente un nombre completo que contenga espacios, por

ejemplo:

Producto Cooperadora

Sin utilizar ignore(), getline() encontraría el salto de línea pendiente y no permitiría ingresar correctamente el nombre.

Actividad N5: Manual Programador

Sofia Salaberry, Martina Araujo, Thiago Goya

5

Funcionamiento de main()

En main() se crea la entidad:

```
EntidadProyecto miEntidad = {0, "Vacio - [Su Nombre y Apellido]",  
0.0f};
```

Luego se llama a la función para cargar los datos:

```
cargarDatos(&miEntidad);
```

Finalmente, se muestran los valores de:

```
miEntidad.id  
miEntidad.nombre  
miEntidad.metrica
```

También se muestra la dirección de memoria de la entidad mediante:

```
cout << &miEntidad << endl;
```

Dirección de memoria

La dirección de memoria se obtiene mediante:

```
&miEntidad
```

Un ejemplo de una dirección de memoria puede ser:

```
0x7ffd1234abcd
```

La dirección cambia en cada ejecución dependiendo del sistema operativo y de la memoria RAM.

Errores comunes

1. Utilizar el punto con un puntero.

Incorrecto:

```
ptr.id
```

Correcto:

```
ptr->id
```

2. Olvidar cin.ignore() antes de utilizar getline().

Actividad N5: Manual Programador

Sofia Salaberry, Martina Araujo, Thiago Goya

6

Se debe utilizar `cin.ignore()` después de ingresar un dato mediante `cin >>` y antes de utilizar `getline()`.

3. Confundir `&` y `*`.

`&miEntidad` obtiene la dirección de memoria de la entidad.

`EntidadProyecto* ptr` declara un puntero.

4. Agregar un miembro al struct sin agregar su carga y visualización.

Cuando se agrega un nuevo miembro a la estructura, también debe contemplarse su ingreso y posterior visualización.

Conclusión

El programa aplica los conceptos de struct, punteros, operador flecha (`->`) y manejo del buffer en C++.

El struct permite agrupar datos, mientras que el puntero trabaja con la dirección de memoria y permite modificar la estructura original desde una función.